Response Letter

We thank the reviewers for their valuable comments. The paper has been carefully revised according to the metareview, and all of the requested changes were implemented. Below is our detailed response to the metareview and each reviewer.

Replies to Metareview

Metareview Comment 1: Correct the description of SEDAR's comment-before-mutation and restore-after-mutation mechanism. Clearly explain the actual methodological differences between MORPH and SEDAR, avoiding misleading characterizations of the latter.
Reply: We have revised the description of SEDAR in Sections I, II, and VI to accurately reflect its mutation-handling mechanism. Specifically, SEDAR temporarily comments out fragments incompatible with the downstream fuzzer's parser during mutation and restores them before executing the mutated SQL on the target DBMS. This mechanism does not permanently remove the original query logic. We have also revised the motivating example in Section II to distinguish the intermediate representation used during mutation from the SQL executed by the target DBMS.

We have clarified the methodological differences between MORPH and SEDAR in Sections II and VI. SEDAR translates statements individually using their referenced source-side sub-schemas, while MORPH uses the AQT to propagate evolving target-side schema and type context across statements. In addition, MORPH uses parser and execution feedback to repair incompatibilities before mutation, whereas SEDAR's comment-and-restore mechanism primarily addresses compatibility with the downstream fuzzer during mutation.

We have added a controlled comparison in Section IV to evaluate the two mutation-handling strategies. For each of the four target DBMSs, we applied both strategies to the same 300 translated seeds and ran GRIFFIN for 24 hours in five independent repetitions. Compared with SEDAR's strategy, MORPH's strategy achieves 27.2%, 15.5%, 4.0%, and 9.4% higher branch coverage on DuckDB, MonetDB, Virtuoso, and ClickHouse, respectively.

Metareview Comment 2: Moderate the novelty claims and better distinguish MORPH from existing SQL translation approaches, particularly SEDAR, RISE, QTRAN, and CrackSQL. Explain more concretely what AQT and its associated mechanisms contribute beyond existing techniques, especially in the context of DBMS fuzzing.
Reply: We have revised Sections I, II, III, and VI so that MORPH is no longer positioned as the first whole-test-case transfer framework or the first approach that adapts transferred seeds to downstream fuzzers. We have also clarified that MORPH builds on established techniques such as structural decomposition, localized translation, and dialect-aware retrieval, and that its contribution lies in coordinating them across an ordered test sequence through AQT. We have revised the related work in Section VI to distinguish MORPH from SEDAR, CrackSQL, RISE, and QTRAN in terms of goals, challenges, methods, and results.

Different goals. CrackSQL and RISE target query migration: they translate one self-contained query or procedural artifact at a time and aim to produce an equivalent target query. QTRAN targets oracle transfer: it translates a query and regenerates its target-side mutant so that the metamorphic relation is preserved for logic-bug detection. SEDAR and MORPH share the goal of trans-

ferring existing test cases into fuzzing seeds. For this goal, a transferred seed must execute on the target DBMS, retain testing-relevant SQL structures, and remain parsable and mutable by the downstream fuzzer.

Different challenges. A fuzzing seed is an ordered sequence of independently executed statements, and later statements may depend on the target-side state produced by earlier statements. For example, in `ALTER TABLE t ADD COLUMN b INT; SELECT b FROM t;`, the `SELECT` is valid only if the translated `ALTER TABLE` has produced the expected column on the target DBMS. When the target dialect changes a type, a constraint, or a DDL form, subsequent statements must follow the translated target state rather than the source state. Translating each statement correctly in isolation is therefore insufficient. Our focus is on maintaining this evolving target-side context across an ordered test sequence, beyond translating a self-contained query or preserving an oracle relation. SEDAR addresses the same setting but translates each statement using a source-side sub-schema and does not maintain translated target-side context across statements.

Different methods. AQT is built on a standard SQL AST; its contribution is not the tree representation itself but the cross-statement information it adds and how this information is consumed during transfer. First, AQT records forward read-after-write dependency edges between statements. These edges identify dependency-relevant preceding statements for context selection during instantiation and repair, while the original statement order is retained. Second, AQT records the cumulative schema context of the source test case, including object-state summaries such as object existence, table definitions, column names and types, constraints, and DDL-induced schema changes. During instantiation, MORPH maintains the corresponding target-side context and updates it after each translated statement. Each statement is thus instantiated and repaired against the translated target state. Third, abstract labels separate dialect-specific constructs from the surrounding structure, so that dialect-aware instantiation remains local while the original structure is preserved. Finally, MORPH uses parser and execution feedback to repair transferred seeds before mutation, whereas SEDAR temporarily comments out parser-incompatible fragments during mutation and restores them afterward. As described in their papers, the workflows of CrackSQL, RISE, and QTRAN do not propagate target-side context across the statements of a test case. We have updated Table I and Sections II and VI to reflect these differences.

Different results. The evaluation in Section IV provides evidence for the practical value of this design. Disabling cross-statement context propagation reduces execution validity from 0.661 to 0.523, supporting the contribution of target-side context propagation. Replacing dependency-based context selection with a recency-based window or the full context reduces execution validity to 0.599 and 0.643, respectively, while increasing input tokens (the reply to **Comment** 1.2). At the system level, MORPH improves execution validity by 130% and branch coverage by 14% over SEDAR, and the controlled mutation-handling comparison is described in the reply to **Metareview Comment** 1.

Metareview Comment 3: Clarify how MORPH differs from and complements SmartFuzz in seed construction. The additional comparison is useful but limited in its experimental scope. The revision should acknowledge these limitations and avoid overstating comparative advantages.
Reply: TODO.

Metareview Comment 4: Clarify the provenance of the SEDAR binary used in the experiments, particularly given conflicting reports about its public availability. Provide sufficient information about the artifact and configuration to support reproducibility.
Reply: We have clarified the provenance of the SEDAR binary in Section IV, stating that the SEDAR binary used in our experiments was obtained from the authors of SEDAR. This binary runs the complete SEDAR pipeline, including schema capture, end-to-end LLM translation, and mutability refinement. To support reproducibility, we have included the SEDAR experimental configuration and all SEDAR-generated seeds used in our experiments in our artifact.

Replies to Reviewer A

Comment 1.1: Can the authors evaluate source-to-target result or state equivalence?
Reply: We have added a source-to-target result and state equivalence study in Section IV. We randomly sampled 100 MySQL test cases and evaluated SQLGlot, RISE, SEDAR, and MORPH on the same cases for transfer to DuckDB. We compared query results and the resulting database states, including schemas and table contents. Translation or execution failures count as non-equivalent, and both rates use all 100 cases as the denominator. MORPH achieves 52.0% result equivalence and 74.0% state equivalence, compared with 14.0%/56.0% for SQLGlot, 34.0%/65.0% for RISE, and 38.0%/69.0% for SEDAR. This study provides supplementary evidence for the sampled source-target pair, without claiming general semantic equivalence. We have also renamed "Semantic Correctness" to "Execution Validity" throughout the paper and clarified its definition in Section IV to distinguish runtime executability from result or state equivalence.

Source-to-target equivalence on 100 randomly sampled MySQL-to-DuckDB test cases. Translation or execution failures count as non-equivalent.

Method	Result Eq. (%)	State Eq. (%)
SQLGlot	14.0	56.0
RISE	34.0	65.0
SEDAR	38.0	69.0
MORPH	**52.0**	**74.0**

Comment 1.2: What independent benefit does the dependency graph provide beyond context propagation?
Reply: Context propagation maintains the target-side state of the test case, while the dependency graph determines which preceding statements are provided as context for the current statement. For each statement, MORPH uses Δ to select its direct predecessors, i.e., the preceding statements that write objects read by the current statement, keeps only the most recent one for each set of shared objects, and provides their source and translated statements in the prompts for instantiation and repair. For example, in `ALTER TABLE t ADD COLUMN b INT; SELECT b FROM t;`, the dependency edge selects the `ALTER TABLE` statement as context when translating and repairing the `SELECT`, regardless of how many unrelated statements appear between them.

To isolate this benefit, we have added an ablation study in Section IV. We compare four context-selection modes under the same LLM, knowledge base, source corpus, repair budget, and statement order: (1) MORPH, which selects dependency predecessors through Δ; (2) MORPH$_{win}$, which

replaces dependency-based selection with the five most recent successfully translated statements while keeping the same object summaries, so that it differs from MORPH only in whether the dependency graph is used; (3) MORPH$_{all}$, which provides all preceding successfully translated statements together with the complete source-side and target-side context; and (4) MORPH$_{w/o\text{-}ctx}$, which disables target-side context propagation. We measure execution validity at the test-case level, the number of predecessor statements provided per statement, the proportion of provided predecessors that are dependency-relevant (i.e., connected to the current statement in Δ), and input tokens per statement.

TABLE VII: Contribution of the dependency graph to context selection.

Mode	Exec.	Preds/Stmt	Relevant Preds (%)	Input Tokens/Stmt
MORPH	**0.661**	0.47	100.0	509.6
MORPH$_{win}$	0.599	1.19	22.6	578.4
MORPH$_{all}$	0.643	27.95	4.2	1,842.3
MORPH$_{w/o\text{-}ctx}$	0.523	0.49	100.0	488.5

As shown in the table, MORPH achieves higher execution validity than MORPH$_{win}$ and MORPH$_{all}$ (0.661 vs. 0.599 and 0.643), while reducing input tokens per statement by 11.9% and 72.3%, respectively. In MORPH$_{win}$ and MORPH$_{all}$, only 22.6% and 4.2% of the provided predecessors are dependency-relevant. In addition, 15.4% of statements select dependency predecessors outside the five most recent statements, which a recency-based window cannot capture. The context of MORPH$_{all}$ grows with test-case length: on test cases with more than 20 statements, its input tokens per statement are 4.98$\times$ those of MORPH. These results show that the independent benefit of the dependency graph lies in selecting a compact, dependency-relevant context whose size does not grow with test-case length, while achieving higher execution validity than larger contexts.

Comment 1.3: How were CrackSQL, RISE, and SEDAR adapted and configured for the four target DBMSs, and how did the authors ensure a fair comparison?
Reply: We have added the configuration of the translation baselines in Section IV, and the number of source test cases used for each target DBMS in the experimental setup. Each baseline follows its original translation workflow. SQLGlot and jOOQ use their built-in dialect support; CrackSQL and RISE use the official SQL documentation of each target DBMS as dialect knowledge, while SEDAR requires no additional dialect knowledge. Since CrackSQL and RISE are designed to translate one query or artifact at a time, we translate the statements of each test case individually and reassemble them in their original order, without schema context. SEDAR translates complete test cases using the extracted source schemas as context. To ensure a fair comparison, all methods use the same source test cases for each target DBMS (5,879, 4,551, 5,850, and 6,990 test cases for MonetDB, Virtuoso, DuckDB, and ClickHouse, respectively), and no method receives additional manually written translation rules.

Comment 1.4: Novelty MORPH should therefore not be positioned as the first whole-test-case transfer framework or the first system to adapt transferred seeds to downstream fuzzers.
Reply: We have revised Sections I, II, III, and VI so that MORPH is no longer positioned as the first whole-test-case transfer framework or the first approach that adapts transferred seeds to downstream fuzzers. We have also clarified the methodological differences between MORPH

and SEDAR. Details can be found in the reply to **Metareview Comment** 1 and the reply to **Metareview Comment** 2.

Comment 1.5: These works show that structural decomposition, localized translation, and dialect-aware retrieval are not individually new. However, none of them models an independently executed stateful fuzzing test sequence, maintains target-side context across its statements, or integrates the result with coverage-guided seed mutation in the manner proposed by MORPH.
Reply: We have clarified in Sections I and VI that MORPH builds on established techniques such as structural decomposition, localized translation, and dialect-aware retrieval, and that its contribution lies in coordinating them across an ordered test sequence through AQT. We have also revised Section VI to distinguish MORPH from CrackSQL, RISE, and QTRAN in terms of their goals. Details can be found in the reply to **Metareview Comment** 2.

Comment 1.6: The related-work discussion contains several details that should be corrected. In particular, the paper states that CrackSQL employs genetic algorithms to mine mutation patterns, but the cited work instead uses functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation. In addition, describing RISE as strictly processing a single SQL statement is too narrow because it also evaluates complete stored procedures containing control flow and multiple DML statements. A more accurate distinction is that RISE processes one self-contained query or procedural artifact at a time rather than a test file containing an ordered sequence of independently executed statements.
Reply: We have corrected the related work in Section VI. CrackSQL is now described as combining functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation. RISE is now described as processing one self-contained query or procedural artifact at a time, rather than an ordered test sequence of independently executed statements.

Comment 1.7: **Technical Clarity** It is therefore unclear what (G_{tc}) contributes independently beyond recording the dependencies and preserving an order that is already present. The reported ablation removes context propagation rather than the graph itself, so it establishes the value of context but not an independent benefit from (G_{tc}). The paper should explain whether the graph affects retrieval, prompting, repair, or any other step not apparent from the algorithms.
Reply: We have revised Algorithm 2 and its description in Section III. We have removed the stable topological sort, since the original statement order already satisfies all forward read-after-write dependencies, and we have made the use of the dependency graph explicit: SELECTCONTEXT uses Δ to select dependency-relevant predecessors, whose source and translated statements are provided in the prompts for instantiation and repair. We have also added an ablation study in Section IV that isolates the dependency graph from context propagation. Details can be found in the reply to **Comment** 1.2.

Comment 1.8: The paper should also define the content and construction of the "object-state summaries" in (D) and clarify which forms of state are represented. The current formal definition explicitly supports forward RAW dependencies and cumulative schema/type context. Broader descriptions such as preserving all schema, data, and state dependencies should be narrowed unless

additional dependency forms are implemented.
Reply: We have defined the object-state summaries in Section III. Each summary records whether a schema object exists, its table definition, column names and types, constraints, and the schema changes induced by preceding DDL statements, and the summaries are updated incrementally in statement order. We have also clarified that $\mathscr{D}$ does not record table contents or other data-level state. In addition, we have revised the broader descriptions in Sections III and IV, e.g., "propagates schema, type, and state context" to "propagates schema and type context", so that they are limited to forward read-after-write dependencies and cumulative schema and type context.

Comment 1.9: Semantic Correctness I suggest renaming this metric to "Execution Validity" or "Runtime Validity" and narrowing statements concerning equivalent syntax, semantic-intent preservation, and high-fidelity transfer. A sampled source-to-target behavioral comparison would provide useful supplementary evidence.
Reply: We have renamed "Semantic Correctness" to "Execution Validity" throughout the paper and clarified its definition in Section IV. We have also revised the description of instantiation in Section III, revising "to guarantee that the generated queries remain semantically consistent with the target dialect" to "to keep the generated queries valid in the target dialect". Section III also states that constant substitution and minimal commenting may change the behavior of transferred seeds. In addition, we have added a sampled source-to-target result and state equivalence study as supplementary evidence. Details can be found in the reply to **Comment** 1.1.

Comment 1.10: Evaluation The evaluation provides strong evidence that the complete MORPH pipeline produces practically useful seeds. In particular, the confirmed bugs and improvements over SEDAR demonstrate that the approach can exercise target DBMS code that remains difficult to reach with existing seeds.

My main concern is the configuration of the translation baselines. The original CrackSQL evaluation focuses on PostgreSQL, MySQL, and Oracle, while RISE evaluates PostgreSQL-to-MySQL and PostgreSQL-to-Oracle translation. MORPH applies these systems to MonetDB, Virtuoso, DuckDB, and ClickHouse. The paper should explain how each baseline was extended to these target dialects, including the dialect documents, knowledge bases, translation rules, LLMs, prompts, and manual effort used. It should also explain how a statement- or artifact-level translator was applied to a complete test sequence and what schema context, if any, was provided.

The SEDAR reproduction also requires clarification. The original SEDAR mutability-refinement algorithm temporarily hides fuzzer-parser-incompatible regions and restores them after mutation. The paper currently summarizes this process as comment-out sanitization and attributes structural simplification to it. The authors should confirm whether their reproduction includes the restoration step. If the motivating example is an observed failure in the reproduced system rather than the general SEDAR algorithm, this distinction should be stated explicitly.

The paper reports average coverage over five runs, but Tables III and VII provide no variance, confidence intervals, or statistical tests. Such information is important for stochastic fuzzing experiments. It is also unclear whether the reported bug counts are unions across the five runs or results from a particular run.

The full bug-finding campaign discovers 28 bugs, whereas the 24-hour comparison reports 22. The duration and computational budget of the full campaign should be reported. Similarly, Table VIII should specify the currency of the cost, the unit of time, and the total offline cost of processing

all test files.

Finally, the component ablation is performed only for PostgreSQL-to-DuckDB transfer. Extending at least the most important ablation, particularly context propagation, to additional source-target pairs would strengthen the generality of the findings.
Reply: TODO.

Comment 1.11: Verifiability and transparency The paper reports the target DBMS versions, corpus size, primary model, temperature, hardware environment, and an anonymous artifact.

For full reproducibility, the paper should additionally document the baseline adaptations, source-target test allocation, repeated-run aggregation procedure, full campaign duration, and validation procedure for automatically generated translation rules. Table IX reports that 136 rules required correction.
Reply: TODO.

Replies to Reviewer B

Comment 2.1: Could the authors clarify whether the reported coverage/bug-finding improvements primarily stem from AQT-based translation or from a larger/richer effective seed corpus? In a controlled experiment where all baselines use exactly the same number and source of initial seeds, how much of the 14% coverage gain over SEDAR remains attributable to AQT?
Reply: TODO.

Comment 2.2: Regarding the characterization of SEDAR's comment-and-restore mechanism—if both MORPH and SEDAR ultimately feed compatible seeds to GRIFFIN, what is the exact difference in their post-mutation handling? Do seeds repaired by MORPH produce semantically equivalent mutated queries, whereas SEDAR's approach loses that equivalence? If not, what is the claimed advantage?
Reply: TODO.

Comment 2.3: Limited novelty. The paper positions AQT as a key contribution, but AQT largely resembles an existing SQL AST or intermediate representation augmented with abstract labels, read/write sets, dependency edges, and schema context. The authors do not clearly articulate what fundamentally new capability AQT provides over existing SQL representations (e.g., JOOQ's parse tree or SQLGlot's AST) or translation systems (CrackSQL, RISE, QTRAN). The distinction from these systems is insufficiently developed.

More importantly, the evaluation lacks compositional baselines such as "existing SQL translator + SEDAR/GRIFFIN" or "AST-based translation + GRIFFIN". Without such ablations, it is difficult to determine whether the contribution is a fundamentally new transfer technique or mainly an engineering integration of existing components. The authors should add such baselines or provide a clearer theoretical argument for why AQT enables capabilities that prior representations cannot achieve.
Reply: TODO.

Comment 2.4: Incomplete discussion of closely related work. SmartFuzz (OOPSLA 2026) also studies high-quality seed generation for DBMS fuzzing and is directly relevant to the problem addressed by this paper. However, the paper neither discusses it sufficiently nor compares against it experimentally. The authors should add a discussion of SmartFuzz in Section VI (Related Work), clarify the differences in approach, and ideally include it as a baseline in the evaluation if feasible.
Reply: TODO.

Comment 2.5: Potentially inaccurate characterization of SEDAR. The paper characterizes SEDAR's comment-and-restore mechanism as "destructive sanitization" that "sacrifices semantic depth." However, SEDAR's mechanism is designed to temporarily hide target-specific constructs from downstream fuzzers' strict parsers during mutation and restore them afterward. This is a pragmatic compatibility strategy, not a destructive removal of query logic.

The authors should revise their characterization to accurately reflect SEDAR's design intent and clarify the actual difference: MORPH performs pre-migration repair to make seeds parser-compatible, whereas SEDAR defers some compatibility handling until after mutation. A fairer comparison would evaluate both approaches under the same downstream fuzzer (e.g., GRIFFIN) and measure the net effect of each strategy on mutation effectiveness.
Reply: TODO.

Comment 2.6: Questionable classification in Table I.
Reply: We acknowledge SQLGlot's multi-statement transpilation, jOOQ's schema metadata tracking, CrackSQL's DML support, and SEDAR's source-side handling of multi-statement dependencies. We have clarified the criteria for the three symbols in Table I (Section II) and revised the entries accordingly. SQLGlot's *Test Case Migration*, jOOQ's *Cross-stmt. Deps.*, and SEDAR's *Cross-stmt. Deps.* entries are now marked as △, while CrackSQL's *DML* entry is marked as ✓.

TABLE I: Capability comparison of existing SQL translation systems for test-case-level DBMS fuzzing seed transfer.

Tool	DDL	DML	DQL	Test Case Migration	Cross-stmt. Deps.
SQLGlot	✓	✓	✓	△	×
jOOQ	✓	✓	✓	×	△
RISE	△	△	✓	×	×
CrackSQL	△	✓	✓	×	×
QTRAN	△	△	✓	×	×
SEDAR	△	△	✓	✓	△
MORPH	✓	✓	✓	✓	✓

Cross-stmt. Deps. denotes explicit tracking of forward read-after-write dependencies and schema/type context across statements. ✓: supported and documented or evaluated; △: partially supported, i.e., supported only for some statements or settings, or through a separate component; ×: not supported.

Comment 2.7: Unclear source of the improvement. The reported coverage and bug-finding improvements may stem from a larger or more diverse effective seed corpus rather than the AQT-based translation technique itself. For a fair comparison with SEDAR, GRIFFIN, and SQUIRREL, all approaches should use exactly the same source seed corpus and produce comparable numbers of final seeds that pass the target parser.

The current experimental setup does not control for seed corpus size. The authors should: (a) report the number of valid seeds generated by each method before fuzzing, (b) normalize the corpus size across baselines, or (c) conduct an experiment where seed counts are equalized. This controlled setting is necessary to isolate the contribution of MORPH's SQL translation technique and demonstrate that improvements are not simply due to differences in seed quantity or diversity.
Reply: TODO.

Comment 2.8: Translation overhead is unclear. Table VIII reports "Cost/TS" and "Time/TS" but does not specify the unit of measurement (e.g., USD per test suite? seconds per test suite?) or the size of the test suite being translated. Additionally, the paper does not compare MORPH's end-to-end translation time and LLM cost with previous methods such as SEDAR.

The authors should: (a) clearly define the units for cost and time, (b) report the average number of statements per test suite, (c) report per-seed translation time and cost, and (d) compare these overhead metrics against SEDAR and other baselines to demonstrate the practical feasibility of MORPH in real-world deployment.
Reply: TODO.

Comment 2.9: Bug evidence is insufficient. The bug links are inaccessible, and the triggering test cases are not publicly available. The artifact includes complete and executable source code.
Reply: TODO.

Replies to Reviewer C

Comment 3.1: The "semantic correctness" metric means only "executes without runtime errors," not result equivalence; a seed that runs but computes something different still counts. Using "+130-450%" as a headline overstates what was measured.
Reply: We have renamed "Semantic Correctness" to "Execution Validity" throughout the paper, including the abstract, the introduction, and Table V. The headline improvements (e.g., 130% over SEDAR and 100%–450% over the baselines) are therefore reported as gains in execution validity rather than semantic correctness. We have also clarified in Section IV that execution validity measures executability on the target DBMS and does not imply result equivalence, and we have added a sampled result and state equivalence study. Details can be found in the reply to **Comment** 1.1.

Comment 3.2: RQ2 baselines are somewhat hard to believe: zero bugs in 24 hours for SQUIRREL, GRIFFIN, and SQLancer across all four DBMSs. The baseline configuration is undocumented, several cells are "-", for unsupported targets, and the "528% over SQLancer" headline is computed over only the two DBMSs that SQLancer supports.
Reply: TODO.

Comment 3.3: Numeric inconsistency: the text claims RISE transfer improved semantic correctness "from 0.314 to 0.776," but Table V's own row reads RISE 0.314 / MORPH 0.748. One of these is wrong.
Reply: We have revised the text in Section IV to state that, in the SQLite-to-DuckDB setting, MORPH improves execution validity from 0.314 (RISE) to 0.748, consistent with Table V. We

have also checked the other numbers reported in Section IV against Table V.

TABLE V: Syntactic correctness and execution validity of transferred seeds.

Method	MonetDB		Virtuoso		DuckDB		ClickHouse	
	Syn.	**Exec.**	**Syn.**	**Exec.**	**Syn.**	**Exec.**	**Syn.**	**Exec.**
No translation	0.589	0.236	0.324	0.097	0.674	0.284	0.613	0.182
SQLGlot	-	-	-	-	0.738	0.186	0.665	0.193
jOOQ	-	-	-	-	0.652	0.136	0.554	0.175
CrackSQL	-	-	-	-	0.740	0.211	0.717	0.264
RISE	0.830	0.266	0.545	0.178	0.857	0.314	0.764	0.278
SEDAR	0.781	0.388	0.562	0.169	0.895	0.365	0.821	0.315
MORPH	**0.929**	**0.776**	**0.670**	**0.629**	**0.939**	**0.748**	**0.895**	**0.694**
vs. SEDAR	+19%	+100%	+19%	+272%	+5%	+105%	+9%	+120%

Syn./Exec.: syntactic correctness/execution validity; "-": unsupported dialects.

Comment 3.4: Missing related work: SQLRight (USENIX Sec 2022) absent from RQ2; sqllogictest is never discussed; classic seed-selection literature ignored.
Reply: TODO.

Comment 3.5: What's novel here is the conjunction of approaches: AST abstraction + RAG + parser-feedback repair, which are established ideas assembled around the AQT.
Reply: TODO.

Comment 3.6: RQ1 (28 bugs, unspecified "full campaign") vs RQ2 (22 bugs in 24h) relationship and campaign duration are unclear.
Reply: TODO.

Comment 3.7: I would recommed tightening the validity claims: add a small differential check on transferred seeds (do they return equivalent results on a shared dialect subset?) to back the "semantic correctness" framing, and lean on coverage/bug counts rather than feature-richness as the primary evidence. For RQ2, publish baseline command lines and seed corpora in the artifact; the all-zero bug rows need more transparency, and SQLRight belongs in the comparison.
Reply: TODO.